Task 2: Food Delivery Orders

BCS | OOP | 40 minutes | Instructor: Dr Muhammad Shahid Bhatti

Study the supplied UML diagram. Implement its Java classes, constructors, methods and relationships exactly as specified.

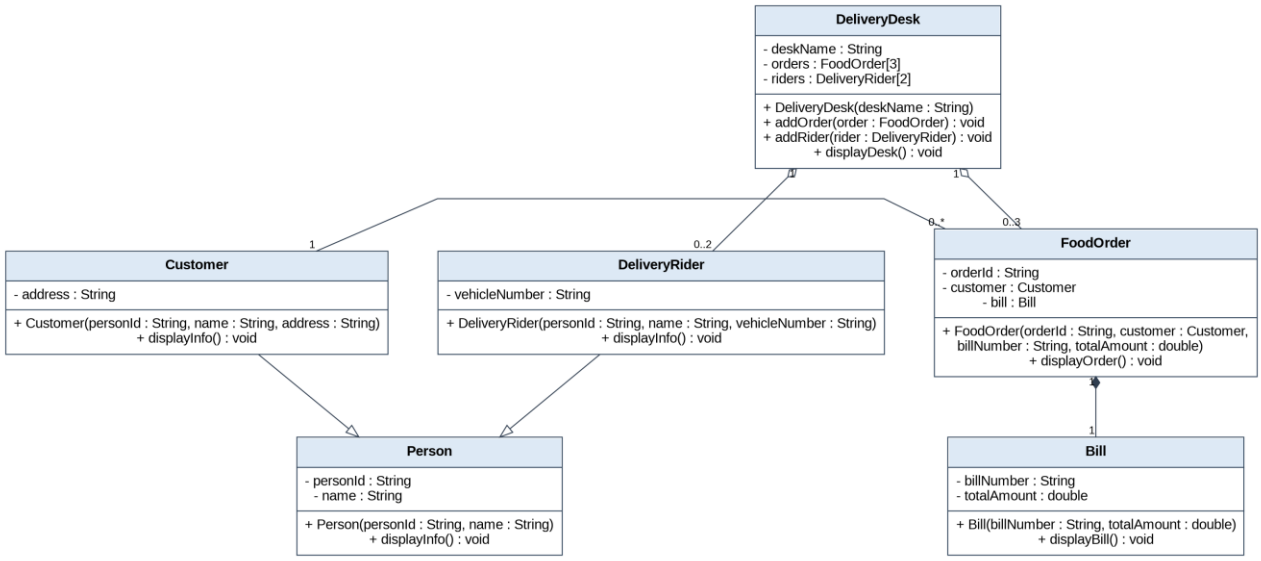
Scenario

A local restaurant delivers meals to customers. Each person in the system has an ID and name. A Customer additionally has a delivery address, while a DeliveryRider has a vehicle number. Both types of people share the same basic information.

Every FoodOrder automatically creates its own Bill when the order is constructed. A Bill stores a bill number and total amount. You do not create a Bill independently in main and then attach it to an order.

A DeliveryDesk manages up to three existing food orders and two existing riders. FoodOrder and DeliveryRider objects are created independently and then registered with the desk. The desk stores references to those objects in ordinary fixed-size arrays.

UML Class Diagram



Read the UML notation: hollow triangle = inheritance; black diamond = composition; white diamond = aggregation; plain line = association. Multiplicities show how many objects participate. The editable PlantUML source is provided separately.

Classes and Attributes

Class	Required attributes
Person	personId, name
Customer extends Person	address
DeliveryRider extends Person	vehicleNumber
Bill	billNumber, totalAmount
FoodOrder	orderId, Customer customer, Bill bill
DeliveryDesk	deskName, FoodOrder[] orders, DeliveryRider[] riders

Required Methods

- Person.displayInfo(); override it in Customer and DeliveryRider.
- FoodOrder.displayOrder(): show order ID, customer details, and bill details.
- DeliveryDesk.addOrder(FoodOrder order), addRider(DeliveryRider rider).
- DeliveryDesk.displayDesk(): display all registered orders and riders.

Constructor instructions: Person initializes its ID and name. Customer and DeliveryRider call super(personId, name). FoodOrder receives the bill number and amount and constructs its own Bill internally. DeliveryDesk(String deskName) creates the two fixed-size arrays with capacities 3 and 2; orders and riders are added later.

What to Create in main()

- Create one Customer and two DeliveryRider objects.
- Create two FoodOrder objects for the customer, each with its own Bill.
- Create one DeliveryDesk and add the orders and riders.
- Display the complete desk information.

Relationships to Demonstrate

Inheritance: Customer and DeliveryRider extend Person. Composition: FoodOrder creates and owns Bill. Aggregation: DeliveryDesk stores existing orders and riders.

Completion Checklist

- ☐ All classes compile and run.
- ☐ Constructors use super(...) in subclasses.
- ☐ Inherited display methods are overridden where specified.
- ☐ Composition objects are constructed within their owner.
- ☐ Aggregation objects are created first and added/passed afterward.
- ☐ The program runs and prints understandable output.

Submission

Save all .java source files for this task in its own folder. At the end of the lab, submit ONE ZIP file containing completed folders for Tasks 1–4. Do not submit four separate ZIP files.